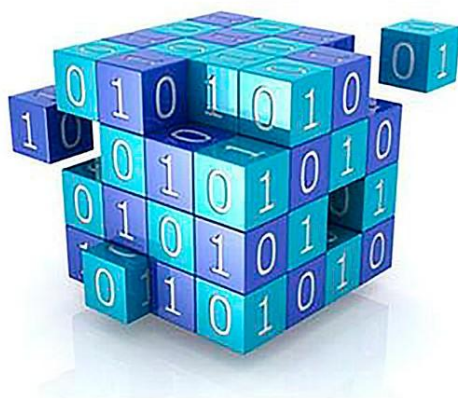




ESTRUCTURA DE DATOS

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:

Manejo de Estructura de
Datos

Unidad #: 3
Semana 13

Fecha: 19-05-2025

Temas: Manejo de Estructura de Datos

Autor: _____

Introducción

Bienvenidos al material de lectura de la semana #13 de la Unidad #3 de la materia Manejo de Estructura de Datos. En este material explicaremos los fundamentos sobre el Manejo de Estructuras de Datos.

Este material de lectura está diseñado para proporcionar una base sólida sobre los fundamentos de programación y análisis de algoritmos, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

¡Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas!

Sección 1: Pilas

Una pila (stack) es una estructura lineal en la que los datos solo se pueden acceder por un extremo, denominado tope o cima. En esta estructura, solo se pueden realizar dos operaciones: añadir un elemento (push) y eliminar un elemento (pop). El último elemento en entrar será el primero en salir, por lo que se le llama estructura LIFO (Last In, First Out).

Cuando se almacena una pila en la memoria de un ordenador, los elementos no se mueven físicamente; solo cambia la posición del tope de la pila, indicada por un puntero de pila. Otro puntero indica la base de la pila, que mantiene el mismo valor mientras la pila exista. Una pila vacía se representa cuando el puntero de pila tiene el mismo valor que la base.

Las pilas son cruciales en computación, utilizadas para calcular expresiones, traducir lenguajes de ordenador y transferir el control entre partes de un programa. Un ejemplo de su uso es en una función que calcula la raíz cuadrada de un número: el argumento se saca de la pila para el cálculo, y el resultado se vuelve a meter en la pila, siguiendo el principio LIFO.

En resumen, una pila es un contenedor de datos homogéneo, secuencial y de tamaño variable, con acceso solo a través de la cabeza de la pila.

La clase Stack

Un objeto de la clase Stack es una pila que permite almacenar objetos y recuperarlos en orden inverso a su inserción, es decir, siempre se recupera el último elemento insertado (LIFO).

Creación e Inserción de Elementos

Para crear una pila vacía en Java

```
Stack pila = new Stack();
```

Para insertar un objeto en la pila se usa el método push. Ejemplo en Java

```
pila.push("Hola");
```

Aquí, el argumento de **push** puede ser de cualquier tipo (int, double, String, boolean).

Recuperación de Elementos

Para recuperar y eliminar el último elemento insertado:

```
println(pila.popString());
```

Este método despliega el string "Hola" y lo elimina de la pila. El tipo debe coincidir con el tipo del valor del elemento insertado.

Para obtener el valor del tope de la pila sin eliminarlo en Java:

```
println(pila.firstString());
```

Este método despliega el string "Hola", pero no lo elimina de la pila.

Uso del Paquete

Es necesario incluir la línea en Java:

```
import tools.*;
```

Constructor

`Stack();` Crea una pila inicialmente vacía.

Métodos

Inserción

- **void push(int val):** Inserta un entero en la pila.
- **void push(double val):** Inserta un valor real en la pila.
- **void push(String s):** Inserta un string en la pila.
- **void push(boolean b):** Inserta un valor booleano en la pila.

Recuperación y Eliminación

- **int popInt():** Recupera y elimina el entero en el tope de la pila.
- **double popDouble():** Recupera y elimina el valor real en el tope de la pila.
- **String popString():** Recupera y elimina el string en el tope de la pila.
- **boolean popBoolean():** Recupera y elimina el valor booleano en el tope de la pila.

Recuperación sin Eliminación

- **int firstInt():** Recupera el entero en el tope de la pila sin eliminarlo.
- **double firstDouble():** Recupera el valor real en el tope de la pila sin eliminarlo.

- **String firstString():** Recupera el string en el tope de la pila sin eliminarlo.
- **boolean firstBoolean():** Recupera el valor booleano en el tope de la pila sin eliminarlo.

Otros Métodos

- **int size():** Devuelve el número de elementos en la pila.
- **boolean isEmpty():** Devuelve verdadero si la pila está vacía, falso en caso contrario.
- **boolean isInt():** Devuelve verdadero si el elemento en el tope de la pila es de tipo int, falso en caso contrario.
- **boolean isDouble():** Devuelve verdadero si el elemento en el tope de la pila es de tipo double o int, falso en caso contrario.
- **boolean isString():** Devuelve verdadero si el elemento en el tope de la pila es de tipo string, falso en caso contrario.

Sección 2: Colas

Definición y Funcionamiento

Una cola es una estructura lineal donde los datos se insertan por un extremo (final o rear) y se eliminan por el otro (frente o front). Funciona bajo el principio FIFO (First In, First Out), es decir, el primer dato en entrar es el primero en salir.

Implementación

- **Vectores:** Utiliza arrays para gestionar la cola.
- **Listas Enlazadas:** Utiliza nodos enlazados para gestionar la cola.

En cualquier implementación, se requieren dos punteros:

- **FRENTE (front):** Indica el primer elemento de la cola.
- **FINAL (rear):** Indica el último elemento de la cola.

Una cola está vacía cuando:

- En implementación dinámica: ambos punteros (f y r) son nulos.
- En implementación estática: ambos punteros coinciden.

Uso de Colas

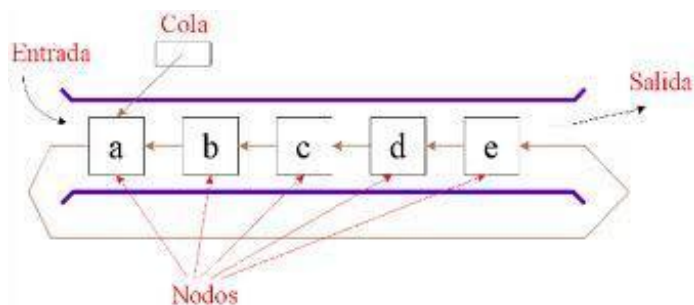
Las colas son útiles para la interconexión y el almacenamiento de datos en tránsito, tanto en redes de ordenadores como en la comunicación entre un procesador y periféricos. Un ejemplo común es la cola de impresión, donde los trabajos se procesan en el orden de llegada.

Comparación con Pilas

Las colas son menos utilizadas que las pilas pero son igualmente importantes. Mientras que una pila sigue el principio LIFO (Last In, First Out), una cola sigue el principio FIFO (First In, First Out).

Analogía

Una cola se puede comparar con un grupo de personas esperando en línea para entrar al cine: la primera persona en llegar es la primera en entrar.



Bibliografía

- Rodríguez L. (2012). Fundamentos de Programación II. Universidad Pontificia de Salamanca (campus Madrid): Escuela Superior de Ingeniería y Arquitectura.
- Rodríguez L. 2012. Fundamentos de programación II. Universidad Pontificia de Salamanca.